Software Engineering :

Implementation and Development

Course prepared by Professor Imed Bouchrika
Software Engineering, ENSIA, 2024-2025

Outline :

- Introduction : Implementation and Dev.
- Programming Best Practices : Clean Code
- Refactoring
- Technological Tools
  - Programming Languages
  - Development Environment
  - Frameworks

Introduction: Implementation & Development

- Is the development stage just a Programming task where the code is written ?
- Is the Programming Task just an execution of what the software architects designed and envisioned ?
- Why can't just the software developers or architects write the code themselves directly based on how they designed the system ?

Introduction: Implementation & Development

- Is the development stage just a Programming task where the code is written ?
    - The major task of the development state is coding, though, it includes other activities including :
        - Development environment setup and administration
        - Data Schema or Database Design and Implementation
        - Communication/API/Network Schema Design
        - Content creation
        - UI and Graphics design

Introduction: Implementation & Development

- Is the Programming Task an execution of what the software architects designed and envisioned ?
    - Programming is a creative process to write code in order to implement the designs for a given software system. It will involves solving problems and taking design decision at the low-level

Introduction: Implementation & Development

- Why can't just the software developers or architects write the code themselves directly based on how they designed the system ?
  - Because ?

Introduction: Implementation & Development

- Programming activities includes :
  - Thinking Code
  - Writing Code
  - Reading Code
  - Reviewing Code
  - Testing Code
  - Fixing Bugs in Code
  - Compiling Code

Introduction: Implementation & Development

- Programming activities includes :
  - Thinking Code
  - Writing Code
  - Reading Code
  - Reviewing Code
  - Testing Code
  - Fixing Bugs in Code
  - Compiling Code

Which task is more difficult and take more time

Introduction: Implementation & Development

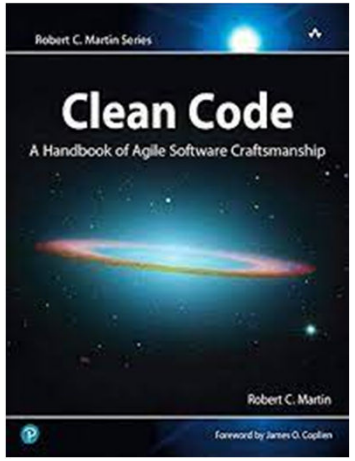
- Reading Code:

  - “ Indeed, the ratio of time spent reading versus writing is well **over 10 to 1**. We are constantly reading old code as part of the effort to write new code. ...[Therefore,] making it easy to read makes it easier to write. ”

    Robert C. Martin (Uncle Bob) “Clean Code: A Handbook of Agile Software Craftsmanship”

Introduction: Implementation & Development

- *Robert C. Martin (Uncle Bob) “Clean Code: A Handbook of Agile Software Craftsmanship”*



Introduction: Implementation & Development

- The problem in development :
  - Programmers can spend :
    - Only few minutes per day coding
    - Hours of paid time reading old code per day (can reach 5hours/day)
  - This is because ?

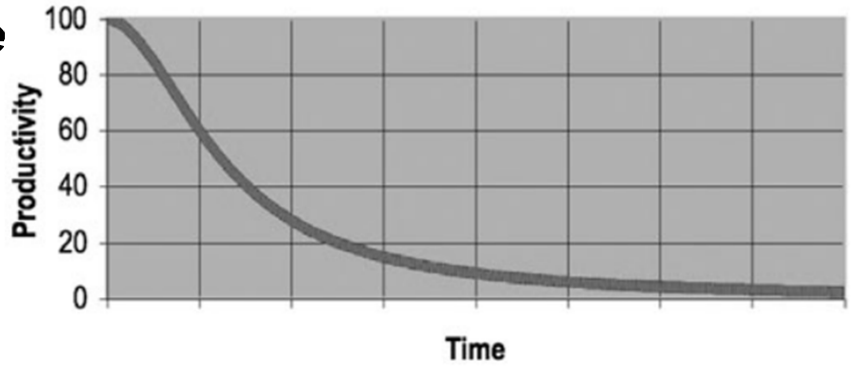
Introduction: Implementation & Development

- The problem in development :
  - Programmers can spend :
    - Only few minutes per day coding
    - Hours of paid time reading old code per day (can reach 5 hours/day)

Aim : Write readable and clean code to reduce time wasted on understanding previous code

Programming Principles: Clean Code

- The impact of writing Spaghetti or bad **Code**
  - Less productivity
  - Cost money and resources
  - Professional survival



  - Even if you add more staff to the team, there will no be difference
  - Risk going into a cyclic loop “Do the grand redesign → bad code again → redesign → .Failure of the project
  - Not maintainable

Programming Principles: Clean Code

- Why programmers make a messy code :

Programming Principles: Clean Code

- Why programmers make a messy code **:**
  - **Important and tight deadline ?**
    - Making messy code, would just slow more the delivery of your software product

Programming Principles: Clean Code

- Why programmers make a messy code **:**
  - **Lack of experience in Software Development**
    - Young programmers should be trained on the principles and best practices of clean code.
    - Accompanied by Senior Programmers
    - Pair Programming shall be used.
    - Code Review and Inspection must be always observed with constructive feedback.
    - Company should explicitly produce their internal documents on their recommended principles of clean coding

Programming Principles: Clean Code

- Why programmers make a messy code **:**
  - **Being Lazy and careless**
    - Companies should fire careless coders
    - Programmers should learn about ethics and risks encountered for a software badly written.
  - **Not professional**
    - Think that coding is getting a functional software.

Programming Principles: Clean Code

- What's **Clean Code**
  - Defined by **Bjarne Stroustrup** (Inventor of C++) as

"I like my code to be elegant and efficient. The logic should be straightforward to make it hard for bugs to hide, the dependencies minimal to ease maintenance, error handling complete according to an articulated strategy, and performance close to optimal so as not to tempt people to make the code messy with unprincipled optimizations. Clean code does one thing well."

Programming Principles: Clean Code

- What's **Clean Code**
  - Defined by "Big" Dave Thomas (Founder of OTI) as

"Clean code can be read, and enhanced by a developer other than its original author. It has unit and acceptance tests. It has meaningful names. It provides one way rather than many ways for doing one thing. It has minimal dependencies, which are explicitly defined, and provides a clear and minimal API. Code should be literate since depending on the language, not all necessary information can be expressed clearly in code alone.

Programming Principles: Clean Code

- Clean Code aspects to be discussed today :
  - *Meaningful names*
  - *Functions*
  - *Comments*
  - *Formatting and Structuring the code*
  - *Handling of Exception and Errors*

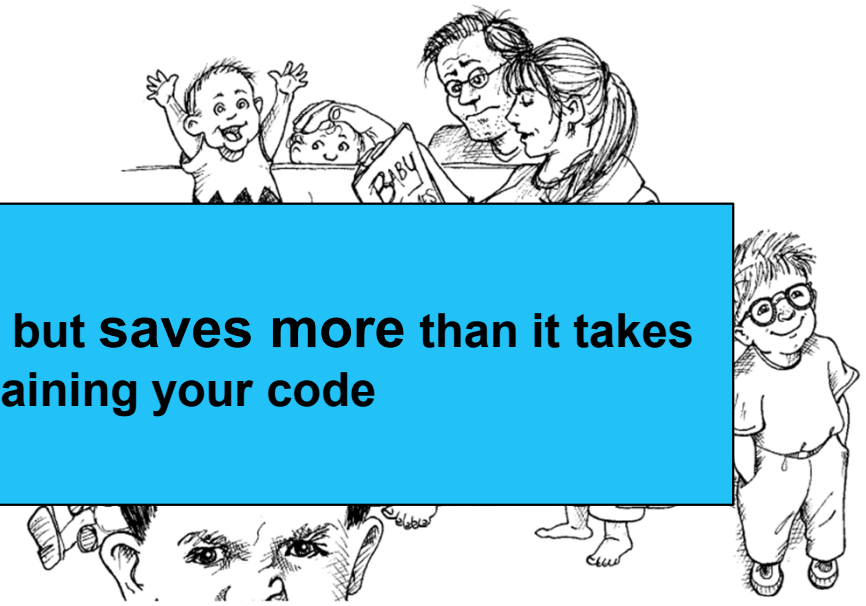
Programming Principles: Clean Code

- Meaningful names
  - “You should name a variable using the same care with which you name a first-born child.”
  - Names are everywhere in software : variables, functions, arguments, classes, packages, components, services, data types…

Programming Principles: Clean Code

- Meaningful names
  - “You should name a variable using t
    r
  - N
    software : variables, functions, arguments, classes, packages, components, services, data types…

Choosing good names takes time but saves more than it takes when reading and maintaining your code



Programming Principles: Clean Code

- Meaningful names
  1. Use Intention-Revealing Names
  2. Avoid Disinformation
  3. Use Pronounceable Names
  4. Use Searchable Names
  5. Avoid Encodings
  6. Don't Be Cute
  7. Conventions for Classes and Methods

Programming Principles: Clean Code

- Meaningful names
  1. **Use Intention-Revealing Names :**
     - If a name requires a comment, then the name does not reveal its intent.
     - The name of a variable, function, or class, should answer all the big questions. It should tell you :
       - Why it exists
       - What it does,
       - How it is used.

Bad

```
int d; // elapsed time in days
```

Good

```
int elapsedTimeInDays;
int daysSinceCreation;
int daysSinceModification;
int fileAgeInDays;
```

Programming Principles: Clean Code

- Meaningful names
  1. **Use Intention-Revealing Names :**
     - Names that reveal intent can make it much easier to understand and change code

What is the purpose of this code?

Bad

```
public List<int[]> getThem() {
        List<int[]> list1 = new ArrayList<int[]>();
        for (int[] x : theList)
                if (x[0] == 4)
                list1.add(x);
        return list1;
}
```

Programming Principles: Clean Code

- Meaningful names
  1. **Use Intention-Revealing Names :**
     - Names that reveal intent can make it much easier to understand and change code.
       - Spacing ?
       - Indentation ?
       - Complexity ?

Bad

```
public List<int[]> getThem() {
        List<int[]> list1 = new ArrayList<int[]>();
        for (int[] x : theList)
                if (x[0] == 4)
                list1.add(x);
        return list1;
}
```

Programming Principles: Clean Code

- Meaningful names
  1. **Use Intention-Revealing Names :**
     - Names that reveal intent can make it much easier to understand

Bad

```
public List<int[]> getThem() {
        List<int[]> list1 = new ArrayList<
        for (int[] x : theList)
                if (x[0] == 4)
                list1.add(x);
        return list1;
}
```

```
public static final STATUS_VALUE=0
public static final FLAGGED=0

public List<int[]> getFlaggedCells() {
    List<int[]> flaggedCells = new ArrayList<int[]>();
    for (int[] cell : gameBoard){
        if (cell[STATUS_VALUE] == FLAGGED){
            flaggedCells.add(cell);
        }
    }
    return flaggedCells;
}
```

Programming Principles: Clean Code

```
int a = 1;
if ( O == l )
  a = O1;
else
  l = 01;
```

- Meaningful names
  2. **Avoid Disinformation**
     - Programmers must avoid leaving false clues that obscure the meaning of code
       - Example : group of bank accounts → accountList
         - The word List may have a meaning to programmers as a data structure, better use the name : accounts, or groupOfAccounts

Programming Principles: Clean Code

- Meaningful names
  3. **Use Pronounceable Names**
     - It would speed up the reading of your code.
     - Help the brain to memorize variable names

```
class DtaRcrd102 {
  private Date genymdhms;
  private Date modymdhms;
  private final String pszqint = "102";
  /* ... */
};
```

```
class Customer {
  private Date generationTimestamp;
  private Date modificationTimestamp;;
  private final String recordId = "102";
  /* ... */
};
```

Programming Principles: Clean Code

- Meaningful names
  4. **Use Searchable Names**
     - Single-letter names and numeric constants have a particular problem in that they are not easy to locate across millions of lines..
       - ***Single-letter names can ONLY be used as local variables inside small functions or methods (or loops)***
     - Example for a variable name that can located easily : WORK_DAYS_PER_WEEK

Programming Principles: Clean Code

- Meaningful names
  5. **Avoid Encodings**
     - Encoding the variable type or scope would :
       - just clutter to your code
       - New employees need to learn about the new encoding
       - Increase the maintenance complexity and risk of your software
     - Examples :
       - Phone

Programming Principles: Clean Code

- Meaningful names
  5. **Avoid Encodings**
     - m_desc : to indicate the m_desc is a member variable
     - phoneString : encoding the type or format into the name

```
private String m_dsc;

PhoneNumber phoneString;
// name not changed when type changed!
```

```
private String description;

PhoneNumber phone;
```

Programming Principles: Clean Code

- Meaningful names
  6. **Don't be cute**
     - Using metaphor is good to remind but never go with names that are only memorable to people who share the author's sense of humor
     - Examples :
       - HolyHandGrenade( ) (for the delete function or unlink)

Programming Principles: Clean Code

- Meaningful names
  7. **Conventions for Classes and Methods**
     - Classes :
       - Should be a noun
       - Capitalized, with every word, starts with capital letter.
     - Methods:
       - Should start with a verb
       - Starts with a lower letter, but capitalized for consecutive words

Programming Principles: Clean Code

- **Functions**
  1. Keep it Small
  2. Do one thing
  3. Function Arguments
  4. Have no side effects
  5. Command Query Separation
  6. DRY : Don't Repeat Yourself
  7. Switch Statements
  8. Exceptions vs. Return Error Codes
  9. Code Structuring

Programming Principles: Clean Code

- Functions
  1. **Keep it Small**
     - You can write functions with
       - 10 lines of code
       - 1000 lines of code
     - Small functions are easier to read, extend, maintain
     - Large functions especially with many deep levels of inner indent (blocks) , it would be impossible to navigate the scope of an if or for ..
     - How small it should be ?

Programming Principles: Clean Code

- Functions
  1. **Keep it Small**
     - You can write functions with

Never bigger than the screen : No more than 100 lines and 120 columns

     - Large functions especially with many deep levels of inner indent (blocks) , it would be impossible to navigate the scope of an if or for ..
     - How small it should be ?

Programming Principles: Clean Code

- Functions
  2. **Do one thing**
     - Functions should be doing it only one task, one action or serve one objective/purpose.
       - A function implemented to perform many tasks:
         - Instructions to Compute data
         - Instructions to Export data to PDF
         - Instructions to Export data to CSV
       - We want to add another column to the data ? without changing the original code.

Programming Principles: Clean Code

- Functions
  2. **Do one thing**
     - Functions should be doing it only one task, one action or serve one

To be done by creating another sub-class with an overriding function
But : you will paste the same code almostly redundantly for a simple addition of a single line.

       - Instructions to Export data to CSV
     - We want to add another column to the data ? without changing the original code.

Programming Principles: Clean Code

- Functions
  2. **Do one thing**
     - Functions shoul… …ne objective/purpo…
       - A function i…
         - Instruct…
         - Instruct…
         - Instruct…
       - We want to… …ging the original …

```
public Data computeData(){
    ....
}
public PDF exportPDF(Data data){
    ...
}
public CSV export CSV(Data csv){
    ...
}
public AbstractReport generateReport(ReportType type){
    Data data=computeData();
    AbstractReport ret_report;
    if (type.code=='PDF'){
        ret_report=exportPDF(data)
    }
    if (type.code=='CSV'){
        ret_report=exportCSV(data)
    }
    return ret_report;
}
```

Programming Principles: Clean Code

- Functions

3. Function Arguments

- Idea number of arguments for a function is zero
- Three arguments, you need to justify
- More than three arguments is **not recommended**,
  - It makes the testing more difficult to consider all possible values for many arguments.
- Use argument objects when possible to reduce the number of args.

```
Circle makeCircle(double x, double y, double radius);
Circle makeCircle(Point center, double radius);
```

Programming Principles: Clean Code

- Functions
  4. **Have no side effects**
     - Function are intended to **do one thing**, but it also does other hidden things. Sometimes it will make unexpected changes to :
       - The variables of its own class.
       - Parameters passed into the function or
       - System globals.

Programming Principles: Clean Code

- Functions

4. **Have no side effects**

```
public class UserValidator {
    private Cryptographer cryptographer;
    public boolean checkPassword(String userName, String password) {
        User user = UserGateway.findByName(userName);
        if (user != User.NULL) {
            String codedPhrase = user.getPhraseEncodedByPassword();
            String phrase = cryptographer.decrypt(codedPhrase, password);
            if ("Valid Password".equals(phrase)) {
                Session.initialize();
                return true;
            }
        }
        return false;
    }
}
```

Programming Principles: Clean Code

- Functions

4. **Have no side**

What if we want to call the function when the user tried to change their password, we ask them again to put their old password to check only ?

Do we need to initialize the session for a logged user ?just to change their password ?

```
public class
    private
    public bo
        User
        if (user != User.NULL) {
            String codedPhrase = user.getPhraseEncodedByPassword();
            String phrase = cryptographer.decrypt(codedPhrase, password);
            if ("Valid Password".equals(phrase)) {
                Session.initialize();
                return true;
            }
        }
        return false;
    }
}
```

Programming Principles: Clean Code

- Functions
  4. **Have no side effects**
     - Function are intended to **do one thing**, but it also does other hidden things. Sometimes it will make unexpected changes to :
       - The variables of its own class.
       - Parameters passed into the function or
       - System globals.

Programming Principles: Clean Code

- Functions
  5. **Command Query Separation**
     - Functions should either
       - **do something** (update the state, variable value…)

         or
       - **answer something** (return the variable value, state…)
     - But **not both**.
     - It can do both ? the aim is to produce readable code with no confusion

Programming Principles: Clean Code

- Functions
  6. **DRY : Don't Repeat Yourself**
     - Avoid duplication
     - Lines of code doing the same business logic, encapsulate them into a function

Programming Principles: Clean Code

- Functions
  7. **Switch Statements**
     - Avoid them in case there is a risk that the number of cases would grow, use the Polymorphism when necessary.

Programming Principles: Clean Code

- Functions
  8. **Exception vs. Return Error Codes**
     - Returning error codes from **command function**s is a subtle violation of command query separation.

Programming Principles: Clean Code

- Functions

8. **Exception vs. Return Error Codes**

```
if (deletePage(page) == E_OK) {
    if (registry.deleteReference(page.name) == E_OK) {
        if (configKeys.deleteKey(page.name.makeKey()) == E_OK){
            logger.log("page deleted");
        } else {
            logger.log("configKey not deleted");
        }
    } else {
        logger.log("deleteReference from registry failed");
    }
} else {
    logger.log("delete failed");
    return E_ERROR;
}
```

Programming Principles: Clean Code

- Functions

8. **Exception vs. Return Error Codes**

```
    try {
        deletePage(page);
        registry.deleteReference(page.name);
        configKeys.deleteKey(page.name.makeKey());
    }
    catch (Exception e) {
        logger.log(e.getMessage());
    }

...

 public void deletePage(Page page) throws SomeException{

 }
```

Programming Principles: Clean Code

- **Comments**
  - Comments are meant to explain complex code but clean and nicely written code.
    - For bad code, “Don’t comment bad code—rewrite it.”
  - The proper use of comments is to compensate for our failure to express ourselves in code
    - A number of software developers considered comments as “necessary evil”
  - Bad Comments vs. Good Comments

Programming Principles: Clean Code

- **Comments**
  - Comments are meant to explain complex code but clean and nicely written code.
    - For bad code, “Don’t comment bad code—rewrite it.”
  - The proper use of comments is to compensate for our failure to express ourselves in code
    - A number of software developers considered comments as “necessary evil”
  - Always Explain yourself in code

Programming Principles: Clean Code

- **Comments**
  - Always Explain yourself in code

```
// Check to see if the employee is eligible for full benefits
if ((employee.flags & HOURLY_FLAG) &&
    (employee.age > 65))
```

```
if (employee.isEligibleForFullBenefits())
```

Programming Principles: Clean Code

- Comments
  - **Good Comments**
    - Legal Comments
      - Showing the author, company, copyright, license information

```
// Copyright (C) 2003,2004,2005 by Object Mentor, Inc. All rights reserved.
// Released under the terms of the GNU General Public License version 2 or later.
```

Programming Principles: Clean Code

- Comments
  - **Good Comments**
    - Informative Comments
      - Showing further/extra information to assist the code reader better understand complex lines of code

```
// format matched kk:mm:ss EEE, MMM dd, yyyy
Pattern timeMatcher = Pattern.compile(
  "\\d*:\\d*:\\d* \\w*, \\w* \\d*, \\d*");
```

Programming Principles: Clean Code

- Comments
  - **Good Comments**
    - Explanation of Intent
      - To explain why a programming decision is made

Programming Principles: Clean Code

- Comments
  - **Bad Comments**
    - Redundant and non-informative comments
    - Misleading comments
    - Mandated Comments which just add clutter to the code
    - Commented-Out Code
    - ...

Programming Principles: Clean Code

- Comments
  - **Bad Com**
    - Redu
    - Misle
    - Manc
    - Comr
    - …

```
/**
 *
 * @param title The title of the CD
 * @param author The author of the CD
 * @param tracks The number of tracks on the CD
 * @param durationInMinutes The duration of the CD in minutes
 */
public void addCD(String title, String author,
                  int tracks, int durationInMinutes) {
  CD cd = new CD();
  cd.title = title;
  cd.author = author;
  cd.tracks = tracks;
  cd.duration = duration;
  cdList.add(cd);
}
```

Programming Principles: Clean Code

- Comments
  - **Bad Comments**
    - Redundant and non-informative co
    - Misleading comments
    - Mandated Comments which just ac
    - Commented-Out Code
    - …

```
/** The name. */
private String name;

/** The version. */
private String version;

/** The licenceName. */
private String licenceName;

/** The version. */
private String info;
```

Programming Principles: Clean Code

- **Formatting and Structuring the code**
  - The functionality that you create today has a good chance of changing in the next release, but the readability of your code will have a profound effect on all the changes that will ever be made

  - Your style of formatting reflect your professionality level.

Programming Principles: Clean Code

- Formatting and Structuring the code
  - **Vertical Formatting**
    - Reduce the vertical distance
      - Declare variables close where they are used. (Exception to the debatable issue for instance variables that they should be placed at the top)
      - Dependent Functions : If one function calls another, they should be vertically close, and the caller should be above the callee,

Programming Principles: Clean Code

- Formatting and Structuring the code
  - **Horizontal Formatting**
    - Number of columns (no more than 120 characters)
    - Use Horizontal Alignment
    - Strict use of Indentation

```
public class FitNesseExpediter implements ResponseSender
{
  private   Socket          socket;
  private   InputStream     input;
  private   OutputStream    output;
  private   Request         request;
  private   Response        response;
  private   FitNesseContext context;
  protected long            requestParsingTimeLimit;
  private   long            requestProgress;
  private   long            requestParsingDeadline;
  private   boolean         hasError;

  public FitNesseExpediter(Socket          s,
                           FitNesseContext context) throws Exception
  {
    this.context =           context;
    socket =                 s;
    input =                  s.getInputStream();
    output =                 s.getOutputStream();
    requestParsingTimeLimit = 10000;
  }
```

Programming Principles: Clean Code

- **Handling of Exceptions and Errors**
  - Employ defensive programming approach where to anticipate for possible errors, invalid data or illegal cases to treat them.

  - Use Exception and handle all those errors whilst **ensuring**
    - **The that your software continue to flow normally**
    - **The data is always in a consistent state after handling an exception**
  - **Never EVER trust the End-User, Expect all evil stuff from them.**

Clean Code : Programming Principles

- How to detect that code is bad (Code smells) ?

Clean Code : Programming Principles

- For Clean Code, it should be :
    - *Code are written for people not machines..*
    - *it's Focused*
    - *Readable, simple and direct with no clutter*
    - *Compact and literate*
    - *Contains only what is necessary*
    - *Makes it easy for other developers to enhance it*
    - *Looks like it's author cares*
    - *Contains no duplicates*

Refactoring of Software Code

- Definition :
  - Refactoring is the process of altering source code whilst keeping its existing functionality unchanged.
- Why Refactoring :
  - To improve readability and maintainability
  - To account for new enhancements as design evolves for new requirements.

Refactoring of Software Code

- What types of changes are involved in the refactoring process.
  - Renaming (variables, classes, methods …)
  - Promoting attribute to class
  - Composing methods
  - Moving features (variables or methods) between classes/objects
  - Organizing and Simplifying code
  - Big Refactoring
- Note that most IDEs offer wizards and tools to assist with the refactoring.

Refactoring of Software

- What types of c
  - Renaming
  - Promoting a
  - Composing
  - Moving feat
  - Organizing
  - Big Refacto